

OMNICART MARKETPLACE PROGRAM

Functional Requirements Document

Detailed functional decomposition of catalog, cart, checkout, payment gateway, refund, reconciliation, security, and operational workflows.

Project:	OmniCart Commerce Platform with Multi-Gateway Payments
Prepared For:	Product, Engineering, QA, Security, Operations, Finance
Generated:	2026-05-13
Classification:	Synthetic test data for ingestion pipeline validation
Version:	1.0

1. Functional Scope

The platform consists of a customer storefront, seller catalog integration, order management, payment orchestration, fulfillment, returns, support tooling, and finance operations. This document focuses on testable functional behavior and interface contracts.

2. Feature Matrix

Feature	Capabilities	Acceptance Criteria
Product Discovery	Search, filters, sponsored ranking, recommendations, category browse	Results show price, seller, rating, delivery estimate, stock state
Cart	Add, remove, save for later, quantity update, promo application	Cart reflects price, tax, shipping, offer eligibility in real time
Checkout	Address, delivery method, payment method, review order	User cannot place prepaid order without valid payment authorization
Payments	Gateway routing, tokenization, auth, capture, webhook, retry	Each transaction has deterministic state and idempotency
Returns	Eligibility, pickup, replacement, refund	Refund amount follows policy and original payment allocation

3. Checkout Functional Flow

```
Customer selects Buy Now
-> Validate inventory and seller availability
-> Resolve address and delivery promise
-> Price cart with tax, shipping, discounts
-> Create order intent in PENDING_PAYMENT
-> Create payment intent with idempotency key
-> Route to gateway by tender, BIN, risk, health, and cost
-> Receive auth result synchronously or by webhook
-> Confirm order or show recoverable failure
```

4. Payment Functional Requirements

ID	Requirement	Priority	Validation
FR-PAY-001	System shall create a payment intent for every prepaid order intent.	Critical	Payment intent has orderId, amount, currency, tender lines, status CREATED.
FR-PAY-002	System shall support gateway routing rules based on tender type, BIN range, gateway health, merchant configuration, risk score, and transaction amount.	Critical	Routing decision includes rule id and fallback sequence.
FR-PAY-003	System shall send idempotency key in every gateway authorization, capture, refund, and void call.	Critical	Duplicate client calls return the same payment result.
FR-PAY-004	System shall support 3DS challenge for eligible card transactions.	High	Payment status changes to ACTION_REQUIRED and resumes after challenge.
FR-PAY-005	System shall consume gateway webhooks and update payment status even if the customer browser closes.	Critical	Webhook signature verified; event id deduplicated.
FR-PAY-006	System shall allow retry with same order intent but a new payment attempt.	High	Failed attempt preserved; successful retry confirms the order.
FR-PAY-007	System shall support partial refunds against order items and shipment lines.	High	Refund amount cannot exceed captured amount minus prior refunds.

5. API Contracts

Create Payment Intent

```

POST /api/v1/payments/intents
Headers:
  Authorization: Bearer <jwt>
  Idempotency-Key: payint_<orderId>_<attemptNo>
Request:
{
  "orderId": "ordint_900121",
  "customerId": "cust_44120",
  "amount": 12999.00,
  "currency": "INR",
  "tenders": [
    { "type": "GIFT_CARD", "amount": 500.00 },
    { "type": "CARD", "amount": 12499.00, "tokenRef": "tok_card_8821" }
  ],
  "riskContext": { "deviceId": "dev_abc", "ipCountry": "IN", "score": 31 }
}
Response 201:
{
  "paymentIntentId": "pi_781120",
  "status": "CREATED",
  "nextAction": "AUTHORIZE",
  "expiresAt": "2026-05-13T12:30:00Z"
}

```

Authorize Payment

```
POST /api/v1/payments/{paymentIntentId}/authorize
Request:
{
  "paymentMethod": "CARD",
  "gatewayPreference": "AUTO",
  "browserInfo": {"userAgent": "...", "acceptHeader": "..."},
  "returnUrl": "https://omnicart.example/checkout/return"
}
Response 200:
{
  "paymentIntentId": "pi_781120",
  "attemptId": "pa_002",
  "status": "AUTHORIZED",
  "gateway": "RAZORPAY",
  "gatewayTransactionId": "rzp_7781",
  "authorizedAmount": 12499.00
}
```

Gateway Webhook

```
POST /api/v1/payments/webhooks/{gatewayCode}
Headers:
  X-Gateway-Signature: sha256=...
Request:
{
  "eventId": "evt_991",
  "eventType": "payment.authorized",
  "gatewayTransactionId": "rzp_7781",
  "amount": 12499.00,
  "currency": "INR",
  "occurredAt": "2026-05-13T12:15:20Z"
}
Response 202: {"accepted": true, "dedupeKey": "RAZORPAY:evt_991"}
```

6. Database Contracts

Table	Key Columns	Purpose	Constraints
payment_intents	id, order_intent_id, customer_id, amount, currency, status	Payment lifecycle aggregate	unique(order_intent_id)
payment_attempts	id, payment_intent_id, gateway, status, gateway_txn_id, failure_code	Individual gateway attempt	unique(gateway, gateway_txn_id)
payment_ledger_entries	id, payment_intent_id, entry_type, amount, tender_type	Finance-grade debit/credit entries	sum entries must balance by payment intent
gateway_webhook_events	id, gateway, event_id, signature_valid, processed_at	Webhook dedupe and audit	unique(gateway, event_id)
refund_requests	id, order_id, amount, reason, status, gateway_refund_id	Refund workflow	amount <= refundable balance

7. Status Model

```
CREATED -> AUTHORIZING -> ACTION_REQUIRED -> AUTHORIZED -> CAPTURED -> SETTLED  
  \-> FAILED  
AUTHORIZED -> VOIDED  
CAPTURED -> REFUND_PENDING -> REFUNDED / PARTIALLY_REFUNDED  
Any state + duplicate webhook -> no-op with audit entry
```

8. Negative and Exception Requirements

- Expired payment intent must reject authorization with HTTP 409.
- Gateway timeout must move attempt to UNKNOWN and trigger status inquiry before user-visible failure.
- Webhook with invalid signature must be rejected and security-audited.
- Payment capture for an unshipped marketplace order must remain blocked unless policy says immediate capture.
- Refund against a chargeback-disputed transaction must require manual finance approval.